

Subtraction Games Can Count and Compute: Exponentially Long Periods, a Counterexample to Guy’s Conjecture, and Universality with a Bounded Number of Moves

September 2026

Abstract

In the subtraction game with a finite move set $S \subset \mathbb{N}_{>0}$, a heap of n tokens is reduced by some $s \in S$ per move and the player who cannot move loses. The sequence of winning and losing positions is eventually periodic, and a long-standing question, going back to R. K. Guy, asks how long the period can be. Guy conjectured that the period is bounded by a polynomial of degree $\binom{n}{2}$ in the largest move when $|S| = n$; Flammenkamp (1997) conjectured, from experiments, that exponential periods occur. Whether super-polynomial periods are possible at all has remained open (Miklós–Post 2024, Larsson–Saha 2024, Larsson–Manabe 2026).

We settle the question. For every $k \geq 2$ we give an explicit set S_k of exactly 26 moves with $\max S_k = 260k - 16$ whose outcome sequence has least period exactly

$$260 \cdot k \cdot 2^k = (\max S_k + 16) \cdot 2^{(\max S_k + 16)/260}.$$

So periods of subtraction games with a bounded number of moves grow exponentially in the largest move: Guy’s conjecture is false, for outcome sequences and for nim-sequences, and Flammenkamp’s conjecture is true. The game is a nonlinear feedback shift register whose feedback is a NOR gate; by laying out clocks, wires and covering residues in blocks of length 260 we make it run a bit-serial binary counter. The layout was found by computer search (Sidon-type constructions, simulated annealing and exact constraint programming, reducing the number of moves $1061 \rightarrow 169 \rightarrow 54 \rightarrow 38 \rightarrow 29 \rightarrow 26$), and every step of the proof is formally verified in Lean 4 with Mathlib.

We then show that one-heap subtraction games are computationally universal *even with a bounded number of moves*: for every cellular automaton F there is a fixed list of move templates whose instances S_L , one for each ring length L , simulate F on every initial configuration. Consequently, deciding whether a heap size n (in binary) is a P-position is PSPACE-complete already for games with a bounded number of moves written in unary—indeed for a single explicit one-parameter family of games—and EXPTIME-complete for a bounded number of moves written in binary, so that in the binary setting no polynomial-time algorithm exists, unconditionally. The universality theorem, the simulation of Turing machines by bounded-move games, and the layout lemma behind them are formally verified as well.

1 Introduction

Let $S \subset \mathbb{N}_{>0}$ be finite. In the *subtraction game* S a position is a heap of $n \geq 0$ tokens; a move removes $s \in S$ tokens provided $s \leq n$; under normal play the player unable to move loses. A position is a *P-position* (a win for the Previous player) iff every move leads to a non-P-position:

$$n \in \mathcal{P}_S \iff (n - s \notin \mathcal{P}_S \text{ for every } s \in S \text{ with } s \leq n). \quad (1)$$

Equivalently, $\mathcal{P}_S$ is the *greedy* subset of $\mathbb{N}$ containing no two elements whose difference lies in S . Writing $w(n) = 1$ if $n \in \mathcal{P}_S$ and $w(n) = 0$ otherwise, (1) reads

$$w(n) = \text{NOR}(w(n-s) : s \in S), \quad w(\text{negative}) = 0.$$

Thus w is generated by a shift register of length $m = \max S$ with a NOR feedback function. By the pigeonhole principle it is eventually periodic, with period at most 2^m .

Subtraction games are the first problem in Nowakowski's *Unsolved Problems in Combinatorial Games* [11]:

“In general, period lengths can be surprisingly long, and it has been suggested that they could be super-polynomial in terms of the size of the subtraction set. However, Guy conjectures that they are bounded by polynomials of degree at most $\binom{n}{2}$ in s_n , the largest member of a subtraction set of cardinality n .”

Althöfer and Bühlmann [1] found families with cubic period and asked whether super-polynomial periods occur for $|S| \geq 5$. In his thesis, Flammenkamp [3] computed record periods of “max-symmetric” five-move sets and observed growth like $2^{0.3 \max S}$; his conjecture that exponential periods exist is Conjecture 1 in the survey of Larsson and Saha [4] and is restated as open by Larsson and Manabe in 2026 [5]. Miklós and Post [10] write that “it is a long time open question if exponential period length is possible”; they proved super-polynomial periods $e^{\Omega(\sqrt{\max S})}$ for a *modified* game in which the outcomes of the first positions are prescribed by a seed. For the actual game no super-polynomial example was known.

1.1 Main results

Theorem 1 (Exponential periods). *There is an explicit family of sets $S_k \subset \mathbb{N}_{>0}$ ($k \geq 2$) such that*

(i) $|S_k| = 26$ for every k : the 23 moves 9, 11, 13, $\dots$, 249 listed in Section 4 together with $260(k-1) + 108$, $260(k-1) + 136$ and $260(k-1) + 244$;

(ii) $\max S_k = 260k - 16$;

(iii) the outcome sequence of the subtraction game S_k has least (eventual) period exactly

$$P_k = 260 \cdot k \cdot 2^k = (\max S_k + 16) \cdot 2^{(\max S_k + 16)/260},$$

in particular $P_k > 2^{\max S_k/260}$. The nim-sequence of S_k has a least period divisible by P_k .

Corollary 2 (Guy's conjecture fails). *For every $C, d \geq 1$ there is a subtraction set S with exactly 26 moves whose outcome period (and nim-period) exceeds $C(\max S)^d$. Concretely, S_{6719} has 26 moves, largest move 1 746 924, and period $260 \cdot 6719 \cdot 2^{6719} > (\max S)^{\binom{26}{2}} = (\max S)^{325}$, contradicting Guy's bound for $n = 26$; $k = 6719$ is the least k for which this happens.*

Adding moves that are “invisible” to the construction (Remark 20) gives the same conclusion for every $n \geq 26$ moves; this is also formally verified. Since games with one or two moves have periods at most $2 \max S$ (a classical fact), the question is now open only in the range between these cases:

Open Problem 3 (The gap between 3 and 25 moves). *Is there a number n with $3 \leq n \leq 25$ such that the periods of n -move subtraction games are not bounded by any polynomial in the largest move?*

Flammenkamp’s experiments suggest that the answer is yes already for $n = 5$. We state Open Problem 3 formally in Lean (`SubGame.OpenGap`), next to the formal proofs that the answer is yes for every $n \geq 26$ (`superPoly_of_ge`) and that Guy’s bound fails for every $n \geq 26$ (`guy_false_of_ge`).

Theorem 4 (Universality with a bounded number of moves; Section 8.2). *For every cellular automaton F with finitely many states there are a block length B and a finite list of move templates (a_t, c_t, δ_t) such that, for every ring length $L \geq 2$, the subtraction game with moves $S_L = \{(a_t L + c_t)B + \delta_t\}$ runs F on a ring of L cells, one after another, on every initial configuration: the outcome of position $((j2^L + T)L + i)B + \rho$ is a bit of the state of cell i after T steps from the j -th configuration.*

Theorem 5 (Complexity; Section 9). *Deciding whether n (in binary) is a P -position of the subtraction game S is*

- (a) *PSPACE-complete when S is given in unary, even if $|S| \leq c$ for a fixed constant c ; in fact there is one explicit family $(S_L)_{L \geq 3}$ with $|S_L| \leq c$ for which the problem “given L in unary and n in binary, is $n \in P_{S_L}$?” is PSPACE-complete;*
- (b) *EXPTIME-complete when S is given in binary, even if $|S| \leq c$. Since $P \neq \text{EXPTIME}$, no polynomial-time algorithm decides the outcome of subtraction games with c moves written in binary.*

Theorem 5 contrasts with the fact that for every *fixed* game the question is trivial (compute the period once), and with games of two moves, whose periods have closed forms.

1.2 What exactly is proved

Because the question has several natural variants, we state precisely which one Theorem 1 answers.

- **The standard game.** One heap, a finite set S of positive integers, normal play, and outcomes defined for every $n \geq 0$ by the standard recursion (1). No seed or prescribed initial outcomes (as in [10]), no modified starting configuration, no additional heaps, no infinite move set, no change of the winning condition.
- **A bounded number of moves.** $|S_k| = 26$ for every k ; only three moves depend on k . Guy’s conjecture concerns a fixed cardinality n , and the theorem gives it for $n = 26$.
- **The standard notion of period.** P_k is the least $p > 0$ with $w(n+p) = w(n)$ for all sufficiently large n (the preperiod is at most 260), for the outcome sequence; for the nim-sequence the least period is a multiple of P_k .
- **Exponential in the largest move.** The growth is not merely exponential in an auxiliary parameter: $P_k = (m+16)2^{(m+16)/260}$ where $m = \max S_k$, so $\log_2 P_k > m/260$.
- **Exact.** The least period is determined exactly, not only bounded from below.
- **Verified.** All of Theorem 1 and Corollary 2, including the exact size, the exact largest move and the explicit instance $k = 6719$, is proved in Lean from the definition of the game; the game was also simulated directly for $k \leq 6$.
- **Not claimed.** We do not determine the least n for which n -move games have super-polynomial periods (Open Problem 3: open for $3 \leq n \leq 25$), nor the best constant in the exponent.

Related work. Larsson and Wästlund [6] and Larsson [9] showed that subtraction games on *two or more* heaps emulate one-dimensional cellular automata (a “time heap” and a “tape heap”), yielding undecidability results; Gurvich and Vyalyi [8] proved PSPACE-hardness and exponential lower bounds for fixed multidimensional games, and related games played on graphs are PSPACE-complete [7]. For one heap, where the outcome sequence is always eventually periodic, we are not aware of any hardness result; recent work on one-heap games gives closed forms for special families [5, 4]. Our constructions pack space and time into a *single* heap by cutting it into blocks.

Formal verification. Theorem 1, Corollary 2, the nim-sequence statement, the General Layout Lemma and the universality theorem are proved in Lean 4 with Mathlib (Section 7), with main statements

- `SubGame.Concrete.main_exact` (Theorem 1: exactly 26 moves, largest move, least period, $P_k > 2^{\max S_k/260}$),
- `Concrete.guy_fails`, `Concrete.not_polynomially_bounded` and `Concrete.guy_explicit` (Corollary 2),
- `SubGame.superPoly_of_ge` and `SubGame.guy_false_of_ge` (every $n \geq 26$ moves), and the formal statement `SubGame.OpenGap` of Open Problem 3,
- `SubGame.Concrete.nim_period_dvd` (nim-sequences),
- `SubGame.GLL.simulate` (General Layout Lemma),
- `SubGame.Univ.universal` (Theorem 14, universality with a bounded number of moves),
- `SubGame.TM.tm_universal` (Corollary 15, every Turing machine with a bounded tape is simulated on all inputs);

The command `#print axioms` reports only `propext`, `Classical.choice` and `Quot.sound`. The complexity theorems are proved on paper on top of these formal results. What is not formalised is the choice of a machine deciding a PSPACE-complete language, the polynomial-time computability of the reduction, and the two-parameter variant used for EXPTIME, which was checked by logic simulation and by simulating the actual games.

1.3 Idea of the construction

The recursion $w(n) = \text{NOR}(w(n-s) : s \in S)$ applies the *same* NOR gate at every position. We cut $\mathbb{N}$ into blocks of length B and give every residue $r \in \{0, \dots, B-1\}$ a role:

- *clocks* $r \in C$: $w(\tau B + r) = 1$ for all blocks τ ;
- *zero residues*: $w(\tau B + r) = 0$, forced by a move landing on a clock;
- *wires* $r = \rho(j)$: the value $w(\tau B + \rho(j))$ is a logical signal $\ell_j(\tau)$.

A move $s = qB + \delta$ applied at a wire at residue e reads residue $(e - \delta) \bmod B$ in an earlier block. If the residues are arranged so that each wire “sees” exactly its intended inputs and never a clock, and clocks see nothing but zero residues, then every wire computes a NOR gate of chosen inputs with chosen delays (Theorem 8). NOR gates are universal, and a move of length about kB gives a delay line of k blocks. We use this to build a bit-serial binary counter whose k -bit word is stored in the delay line: the game counts from 0 to $2^k - 1$ over and over, and the period is exactly $k \cdot 2^k$ blocks. Replacing the counter by a cellular automaton whose cells are processed one per block gives the universality results.

2 The simulation theorem

Definition 6 (Design). A *design* consists of a block length $B \geq 1$, a list $C \subseteq [0, B)$ of clock residues, wire residues $\rho : \{0, \dots, W-1\} \rightarrow [0, B)$ (injective, disjoint from C), a list of *taps* $t = (a, b, \delta)$ with $\delta < B$, and for each wire j a list of inputs (σ, a', b') where σ is a wire or the symbol CLK. For a parameter K the tap t is the move $s_t(K) = (aK + b)B + \delta$ and an input has delay $a'K + b'$. The move set is $S(K) = \{s_t(K)\}$.

For a residue e and tap $t = (a, b, \delta)$ put $\lambda(e, t) = (e - \delta) \bmod B$ (the *landing residue*) and $D(e, t) = aK + b + [e < \delta]$ (the *effective delay*). The key arithmetic fact is:

Lemma 7. *Let $e, \delta < B$ and $s = qB + \delta$. Then $s \leq \tau B + e$ iff $q + [e < \delta] \leq \tau$, and in this case $\tau B + e - s = (\tau - q - [e < \delta])B + \lambda(e, t)$.*

Let $X = C \cup \rho(\{0, \dots, W-1\})$. We require the following *finite* conditions.

- (A) Clocks see only zero residues: $\lambda(c, t) \notin X$ for all $c \in C$ and taps t .
- (B) Wires see only their inputs: if $\lambda(\rho(j), t) \in C$ then $(\text{CLK}, a, b + [\rho(j) < \delta])$ is an input of j ; if $\lambda(\rho(j), t) = \rho(i)$ then $(i, a, b + [\rho(j) < \delta])$ is an input of j .
- (C) Every input of every wire is realised by some tap in this way.
- (D) Every zero residue r has a tap with $a = 0$, $D(r, t) \leq 1$ and $\lambda(r, t) \in C$.
- (E) Every zero residue r has a tap with $a = 0$, $D(r, t) = 0$ and $\lambda(r, t)$ a clock or a wire i with $\ell_i(0) = 1$ (block 0 has no previous block).

Theorem 8 (Simulation theorem). *Let $\ell_j(\tau)$ ($j < W$, $\tau \geq 0$) satisfy the NOR equations*

$$\ell_j(\tau) = 1 \iff \text{every input } (\sigma, a', b') \text{ of } j \text{ with } a'K + b' \leq \tau \text{ has } \ell_\sigma(\tau - a'K - b') = 0,$$

where $\ell_{\text{CLK}} \equiv 1$. If the design satisfies (A)–(E) and all moves are positive, then the P -positions of the subtraction game $S(K)$ are exactly

$$\{\tau B + c : c \in C\} \cup \{\tau B + \rho(j) : \ell_j(\tau) = 1\}.$$

Proof. Let $\mathcal{P}^*$ be the displayed set. By uniqueness of the solution of (1) (strong induction on n , using that all moves are positive) it suffices to show that $\mathcal{P}^*$ satisfies (1). Fix $n = \tau B + e$ with $e < B$. By Lemma 7, the moves $s \leq n$ correspond to taps t with $D(e, t) \leq \tau$, and $n - s$ lies in block $\tau - D(e, t)$ at residue $\lambda(e, t)$. If e is a clock, $n \in \mathcal{P}^*$ and by (A) every $n - s$ lies at a zero residue, so $n - s \notin \mathcal{P}^*$. If e is a zero residue, $n \notin \mathcal{P}^*$, and (D) (for $\tau \geq 1$) or (E) (for $\tau = 0$) provides a move to a position of $\mathcal{P}^*$. If $e = \rho(j)$, then by (B) and (C) the positions $n - s$ that lie in $\mathcal{P}^*$ are exactly those corresponding to inputs of j whose value is 1 (a clock input with delay $\leq \tau$ always has value 1), so (1) at n is precisely the NOR equation for $\ell_j(\tau)$. $\square$

3 A bit-serial binary counter

Fix $k \geq 2$ and write every block index as $\tau = wk + i$ with $0 \leq i < k$. The circuit has eight wires (Figure 1); the notation $x[-d]$ means “the value of x d blocks earlier”.

$$\begin{aligned} \text{boot} &= \text{NOR}(\text{CLK}[-1]) & \text{nm} &= \text{NOR}(\text{boot}, \text{marker}[-k]) \\ \text{marker} &= \text{NOR}(\text{nm}) & \text{ncin} &= \text{NOR}(\text{carry}[-1], \text{marker}) \\ \text{nb} &= \text{NOR}(u[-k]) & g_1 &= \text{NOR}(u[-k], \text{carry}[-1], \text{marker}) \\ \text{carry} &= \text{NOR}(\text{nb}, \text{ncin}) & u &= \text{NOR}(g_1, \text{carry}). \end{aligned}$$

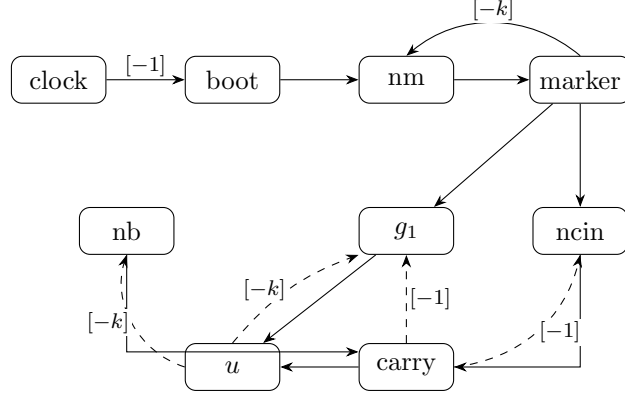


Figure 1: The NOR circuit of the bit-serial counter. Solid edges have delay 0 (same block); dashed edges read an earlier block ($[-1]$: previous block, $[-k]$: the same bit of the previous word).

Lemma 9. Let $\text{low}(w, i) = [w \bmod 2^i = 2^i - 1]$ (the lowest i bits of w are all 1) and let $\text{bit}_i(x)$ denote bit i of x . The unique solution of the NOR equations is

$$\begin{aligned} \text{boot}(\tau) &= [\tau = 0], & \text{marker}(\tau) &= [k \mid \tau], & \text{nm} &= \neg \text{marker}, \\ \text{ncin}(\tau) &= \neg \text{low}(w, i), & \text{nb}(\tau) &= \neg \text{bit}_i(w), & g_1(\tau) &= \neg(\text{bit}_i(w) \vee \text{low}(w, i)), \\ \text{carry}(\tau) &= \text{low}(w, i + 1), & u(\tau) &= \text{bit}_i(w + 1). \end{aligned}$$

In words: word w (blocks $wk, \dots, wk + k - 1$) holds the number $(w + 1) \bmod 2^k$ in binary, least significant bit first.

Proof. Each equation is checked directly. The two arithmetic facts needed are the carry rule $\text{low}(w, i + 1) = \text{bit}_i(w) \wedge \text{low}(w, i)$ and the increment rule $\text{bit}_i(w + 1) = \text{bit}_i(w) \oplus \text{low}(w, i)$; for $u[-k]$ one uses that block $\tau - k$ belongs to word $w - 1$, whose value is w . The carry into bit i is $\text{carry}[-1] \vee \text{marker} = \text{low}(w, i)$. Uniqueness is immediate by induction on τ . $\square$

4 The concrete design

A design implementing the counter with $B = 260$, 29 clocks and 26 taps was found by computer search (Section 6):

clocks C	0, 2, 3, 4, 5, 6, 7, 8, 25, 27, 28, 29, 30, 31, 32, 33, 50, 60, 62, 64, 65, 67, 68, 123, 225, 228, 229, 230, 232
wires ρ	boot 1, nm 10, marker 26, ncin 35, nb 70, g_1 98, carry 147, u 222

With $K = k - 1$ the move set is $S_k = \{(aK + b) \cdot 260 + \delta\}$ over the 26 taps; explicitly

$$S_k = \{9, 11, 13, 16, 51, 72, 75, 77, 84, 89, 104, 110, 112, 124, 135, 148, 156, 171, 176, 186, 209, 211, 249\} \cup \{260(k - 1) + 108, 260(k - 1) + 136, 260(k - 1) + 244\}.$$

The 23 fixed moves are at most $249 < 260 + 108$, so the three k -dependent moves (the long delays feeding nm, nb and g_1) are distinct from them and $|S_k| = 26$ for all $k \geq 2$, with $\max S_k = 260(k - 1) + 244 = 260k - 16$. (In fact the 26 residues δ are distinct, so the moves are distinct for every k .) Conditions (A)–(E) are finite and are verified mechanically. Figure 2 shows the resulting P-positions for $k = 3$.

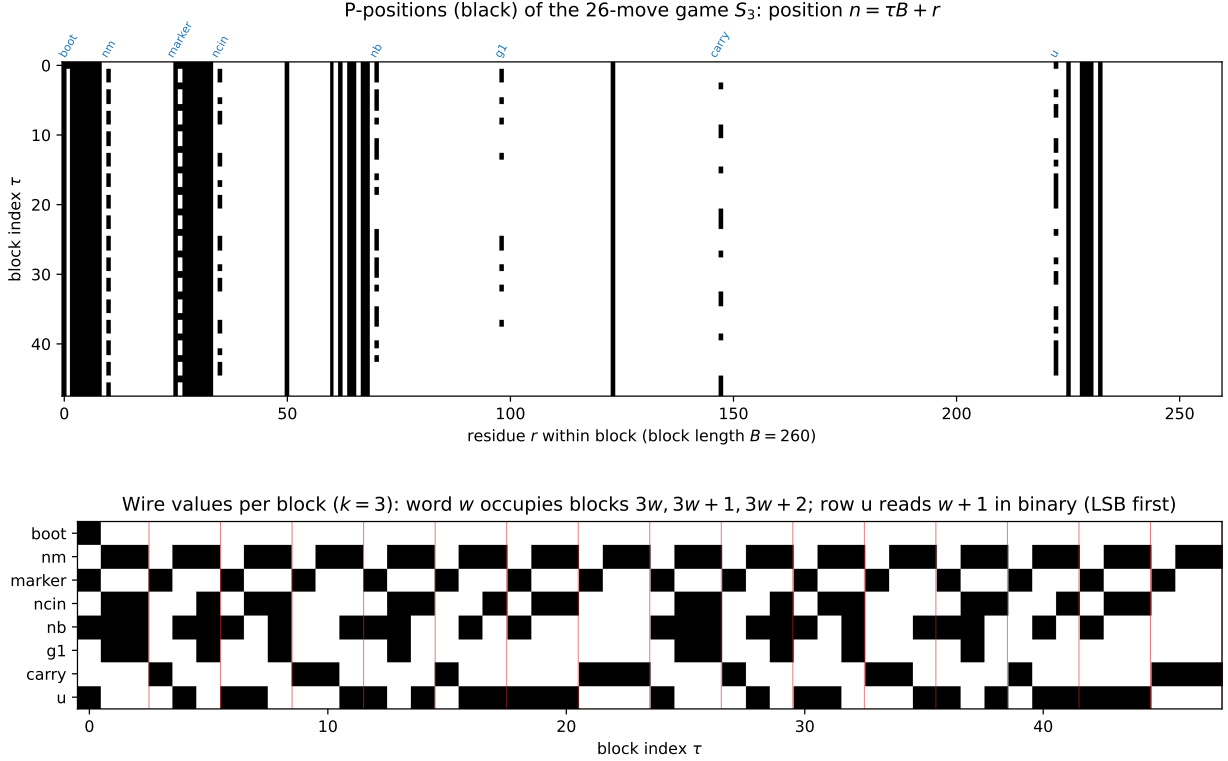


Figure 2: Top: P-positions of the 26-move game S_3 , one block of 260 positions per row. Clocks are the solid columns; the dashed columns are the eight wires. Bottom: the wire values; the u row spells $1, 2, 3, \dots, 7, 0, 1, \dots$ in binary, three blocks per word. The period is $260 \cdot 3 \cdot 2^3 = 6240$.

5 The least period

Theorem 10. *For $k \geq 2$ the outcome sequence of S_k satisfies $w(n + P) = w(n)$ for all $n \geq B$, where $P = B k 2^k$, and every eventual period of w is a multiple of P .*

Proof. Periodicity follows from Theorem 8 and Lemma 9: all closed forms depend only on $\tau \bmod k$ and $(\tau \div k) \bmod 2^k$ except boot, which is constant for $\tau \geq 1$. Conversely let p be an eventual period. (1) If $B \nmid p$, then (a finite check) some clock c is shifted by $p \bmod B$ onto a zero residue, so $w(\tau B + c) = 1 \neq 0 = w(\tau B + c + p)$ for large τ ; hence $p = Bm$. (2) The marker wire gives $[k \mid \tau + m] = [k \mid \tau]$ for all large τ , hence $k \mid m$, say $m = km'$. (3) The u wire gives $\text{bit}_i(w + m' + 1) = \text{bit}_i(w + 1)$ for all $i < k$ and large w , hence $2^k \mid m'$. Thus $P \mid p$. $\square$

Theorem 1 follows, since $\max S_k = 260k - 16$, and Corollary 2 follows because 2^k eventually exceeds $C(260k)^d$.

6 How the design was found

1. **Exploration.** A C program computes outcome periods by cycle detection on the m -bit shift-register state. Record periods of max-symmetric five-move sets grow roughly like $2^{0.33m}$ (confirming Flammenkamp), but along every structured one-parameter family we tried the periods

were erratic—chaotic dynamics that resist proof. This motivated *designing* rather than *finding* long periods.

2. **Layouts.** Random layouts almost never work: small differences between residues collide. Placing wires on a coarse grid $G \cdot Y$ with Y a Sidon set, and clocks in a few residue classes modulo G chosen so that their sums and differences avoid each other, produced the first valid design ($B = 1157, 1061$ moves), confirmed by direct simulation for $k \leq 8$.
3. **Heuristic optimisation.** Choosing covering taps by greedy set cover and running simulated annealing over wire positions, clock sets and B reduced the number of moves $1061 \rightarrow 169 \rightarrow 67 \rightarrow 56 \rightarrow 54$.
4. **Exact optimisation.** For fixed wire positions, conditions (A)–(E) are a Boolean constraint problem in the clock indicators c_r and tap indicators t_δ : “clock sees clock” is the clause $\neg c_r \vee \neg t_\delta \vee \neg c_{r-\delta}$, a zero residue r needs $c_r \vee \bigvee_\delta (t_\delta \wedge c_{r-\delta})$, and so on. Minimising $\sum t_\delta$ with the CP-SAT solver (OR-Tools) found designs with 48 and then 38 moves for the annealed wire placement (the solver’s lower bound for that placement is 24).
5. **Large-neighbourhood search.** Letting every wire move within ± 5 residues of its current position (one Boolean per candidate position, with the pairwise difference constraints written out) and re-solving gave a design with 29 moves at $B = 377$. It uses many more clocks (31 instead of 17), grouped in short runs such as 0–3 and 10–13, so that each covering move serves several zero residues at once, and it leaves every residue above 219 empty.
6. **Shorter blocks.** Re-solving the same model for smaller block lengths, first with the wires fixed and then again with the wires free inside their windows, gave the final design: $B = 260$ and 26 moves, better in both respects.

Every candidate was checked by an exact verifier of conditions (A)–(E), and the final design by simulating the game itself: for $k = 2, \dots, 6$ the simulated P-positions coincide with the counter pattern and the least periods are 2080, 6240, 16640, 41600, 99840. The number of moves and the growth rate can be traded against each other; the designs we found (each checked exactly and by simulating the game for $k \leq 5$) are

block length B	moves	period	growth
377	29	$377 k 2^k$	$2^{\max S/377}$
260	26	$260 k 2^k$	$2^{\max S/260}$
240	28	$240 k 2^k$	$2^{\max S/240}$

The design with $B = 260$ is the one formalised; since `Design.lean` is generated automatically from a design file, the others can be checked in Lean in the same way.

7 Formal verification in Lean

The development (about 3800 lines, Lean 4.33.1, Mathlib) consists of:

<code>Basic.lean</code>	P-set characterisation, uniqueness, computable outcome
<code>Arith.lean</code>	block arithmetic (Lemma 7)
<code>Simulation.lean</code>	designs, conditions (A)–(E), Theorem 8
<code>Bits.lean</code> , <code>Counter.lean</code>	carry/increment rules, Lemma 9
<code>Design.lean</code>	the concrete design; all finite conditions by <code>decide +kernel</code>
<code>Period.lean</code>	Theorem 10
<code>Main.lean</code> , <code>Grundy.lean</code>	Theorem 1, Corollary 2, nim-values
<code>Tests.lean</code> , <code>Exact.lean</code>	sanity checks; exact size and largest move, $k = 6719$
<code>Layout.lean</code>	General Layout Lemma (Lemma 12), <code>GLL.simulate</code>
<code>Transport.lean</code>	the layout lemma for arbitrary finite types of wire names
<code>UnivCore.lean</code>	cellular automata, the universal circuit U_F , its NOR equations
<code>Universal.lean</code>	order and Sidon placement of the wires; Theorem 14
<code>TuringSim.lean</code>	Turing machines, their cellular automata, Corollary 15
<code>OpenProblem.lean</code>	Guy’s bound for n moves, all $n \geq 26$, Open Problem 3

The game semantics are defined independently of the construction: `outcome L n` is defined by well-founded recursion as “no legal move leads to a position with `outcome = true`”, and `grundy L n` as the minimum excludant of the options’ values. The file `Design.lean` is generated from the design found by the search, and the kernel checks conditions (A)–(E) for it. No `sorry`, `native_decide` or additional axioms are used.

8 Universality

8.1 The General Layout Lemma

The counter was laid out by computer search, but a layout exists for *every* NOR circuit.

Definition 11 (Circuit). A *circuit* has wires $0, 1, \dots, W - 1$; every wire j has a finite list $\text{In}(j)$ of inputs (σ, d) , where σ is a wire or CLK and $d = aK + b$ is a delay. We require: (i) $\text{In}(0) = \{(\text{CLK}, 1)\}$ (wire 0 is *boot*); (ii) inputs with delay 0 come from wires of smaller index; (iii) no wire is its own input; (iv) if $\sigma > j$ then $b \geq 1$.

Condition (iii) is necessary: a move that is a multiple of B makes *every* residue read itself, in particular every clock.

Lemma 12 (General Layout Lemma). *Let $Y = \{0 = y_0 < y_1 < \dots < y_{W-1}\}$ be a Sidon set (all differences $y_i - y_j$, $i \neq j$, distinct) with $y_1 \geq 2$, let $B' \geq 3y_{W-1} + 3$ and $B = 13B'$. Place wire j at residue $13y_j$ and clocks at $C = \{1, 5, 12\}$; realise every input (σ, d) of wire j by the move $dB + 13y_j - \rho(\sigma)$ (where $\rho(\text{CLK}) = 5$; for backward edges this is $(d - 1)B + (B + 13y_j - \rho(\sigma))$), and cover the zero residues by the rule of Table 1. Then conditions (A)–(E) hold for every circuit, so by Theorem 8 the subtraction game simulates the circuit. The design has $B = O(W^2)$ and at most $B + 2 + \sum_j |\text{In}(j)|$ moves.*

Proof. Write residues as $13z + \rho$ with class $\rho \in [0, 13)$ and coarse index $z \in [0, B')$. Let X be the set of wire and clock residues. A direct computation gives the classes of the differences $X - X \pmod{B}$, as coarse sets modulo B' :

class	0	1	5	8	12	4, 7, 11	2, 6, 9	3, 10
coarse	$Y - Y$	$(Y - 1) \cup (-Y)$	$-Y$	$Y - 1$	$(Y - 1) \cup (-Y)$	$\{0\}$	$\{-1\}$	$\emptyset$

(for instance class 8 consists of $13y - 5$, wire minus clock 5, with coarse index $y - 1$). Since $Y \subset [0, B'/3)$, these coarse sets do not wrap around.

residue $r = 13z + \rho$, $z \geq 1$	covering clock c (move $r - c$)
$\rho \in \{0, 1, 2, 9\}$	$c = 5$
$\rho \in \{4, 5, 8, 11, 12\}$	$c = 1$
$\rho \in \{3, 7, 10\}$	$c = 1$, except $c = 5, 12, 12$ respectively when $z = B' - 1$
$\rho = 6$	$c = 12$, except $c = 1$ when $z = 1$
residue $r < 13$	moves
$r = 3, 4$	$r - 1$
$r = 2$	$r = 7, 8, 11$: $r - 5$ $r = 9, 10$: $r - 1$
$r = 6$	$B - 3$ (clock 5 of the previous block) and 2 (boot, block 0)
	$B - 6$ (clock 12 of the previous block) and 6 (boot, block 0)

Table 1: Covering the zero residues in the General Layout Lemma.

Intended moves. A wire–wire move has residue $13(y_j - y_i)$. Applied at a clock it lands in the clock’s class with nonzero coarse index, i.e. on a zero residue. Applied at wire m it lands on a wire only if $y_m - y_j + y_i \in Y$; as $Y \subset [0, B'/3]$ this is an equality of integers $y_m + y_i = y_j + y_l$, so the Sidon property forces $m = j$ (the intended target), because $i \neq j$ by (iii). A clock move has residue $13y_j - 5$ (class 8); applied at wire m it lands in class 5 with coarse index $y_m - y_j$, a clock only if $m = j$; applied at a clock c it lands in class $c + 5 \in \{6, 10, 4\}$, which contains no element of X . Hence (B) and (C) hold.

Covering moves. For $z \geq 1$ every clock lies below r , and the move $r - c$ has class $\rho - c$ (coarse index z) or $\rho - c + 13$ (coarse index $z - 1$). Reading off the table of $X - X$, the choices in Table 1 are never differences of two elements of X : e.g. for $\rho = 0$, $c = 5$ gives class 8 with coarse index $z - 1$, which is bad only if $z \in Y$, impossible for a zero residue of class 0; for $\rho = 3$, $c = 1$ gives class 2, bad only if $z = B' - 1$, and then $c = 5$ gives class 11 with coarse index $B' - 2 \neq 0$. The remaining cases are identical in nature and use only $1 \notin Y$, $B' - 1 \notin Y$ and $Y \cap (B' - Y) = \emptyset$. The residues below 13 are checked directly. Since no covering move is a difference of two elements of X , covering moves never make a clock or a wire see anything, which completes (A), and (D), (E) hold by construction (boot has value 1 in block 0). $\square$

Lemma 12 is formally verified in Lean (`Layout.lean`): the proof reduces every condition to linear arithmetic modulo 13 with explicit case splits, discharged by `omega`, plus one application of the Sidon property. Before formalising it we also checked it computationally on 1000 random circuits with up to 40 wires and random Sidon sets.

8.2 Universal games with a bounded number of moves

The General Layout Lemma produces a game whose number of moves grows with the circuit. To make the number of moves *independent of the input* we use a single fixed circuit that enumerates all inputs by itself. The only thing that changes from instance to instance is the length of the delay lines, i.e. the numerical value of a few moves.

Definition 13 (Cellular automata). A b -bit cellular automaton $F = (f, \iota, \beta)$ consists of a local rule $f : Q^3 \rightarrow Q$ on $Q = \{0, 1\}^b$, an initialisation map $\iota : \{0, 1\}^3 \rightarrow Q$ and a boundary state $\beta \in Q$. For a ring length $L \geq 1$, a number $X \geq 0$ and $\Lambda \leq L$, the run $\text{Run}_F(L, \Lambda, X, T, i) \in Q$ ($T \geq 0$, $0 \leq i < L$) is defined by

$$\text{Run}_F(L, \Lambda, X, 0, i) = \iota(v_i, [i = 0], [i = L - 1]), \quad v_i = \begin{cases} \text{bit}_i(X) & i < \Lambda \\ 0 & i \geq \Lambda \end{cases}$$

group	wires (NOR of the listed inputs; $[-d]$ = delay d blocks)	value
framing	boot = NOR(CLK $[-1]$); nm = NOR(boot, mk $[-L]$); mk = NOR(nm) nlc = NOR(mk $[-(L-1)]$), lc = NOR(nlc)	mk = $[i = 0]$ lc = $[i = L - 1]$
timer	the counter of Section 3 with register length L (5 wires)	$u_T = \text{bit}_i(t + 1)$
run start	novf = NOR(boot, tcarry $[-1]$), inj = NOR(nm, novf), hold = NOR(mk, nrs $[-1]$), nrs = NOR(inj, hold), rs = NOR(nrs)	rs = $[T = 0]$
input	a second counter whose carry-in is inj (8 wires)	$u_X = \text{bit}_i(j + 1)$
cells	per state bit q : literals $s_q[-L]$, $s_q[-(L \pm 1)]$, the neighbours masked by mk, lc (boundary β) (9 wires); one NOR per neighbourhood pattern (minterm); $\text{nF}_q = \text{NOR}(\text{minterms with } f_q = 1)$; $t_q = \text{NOR}(\text{rs}, \text{nF}_q)$; eight initialisation terms; ns_q ; $s_q = \text{NOR}(\text{ns}_q)$	$s_q = \text{Run}_F(\dots)_q$

Table 2: The universal enumerating circuit U_F (one parameter). Block $\tau = tL + i$ is cell i of round $t = j2^L + T$. It has $23 + 21b + 2^{3b}$ wires, and its input lists depend on F but not on L .

and $\text{Run}_F(L, \Lambda, X, T + 1, i) = f(c_T(i - 1), c_T(i), c_T(i + 1))$, where $c_T(i) = \text{Run}_F(L, \Lambda, X, T, i)$ and $c_T(-1) = c_T(L) = \beta$ (null boundary). We write $\text{Run}_F(L, X, T, i) = \text{Run}_F(L, L, X, T, i)$.

The initialisation may depend on whether a cell is the first or the last one; this is how a Turing-machine simulation places its head and end markers.

Theorem 14 (Universality with a bounded number of moves). *Let F be a b -bit cellular automaton. There are $B \geq 1$, residues $\rho_0, \dots, \rho_{b-1}$ and a finite list of taps (a_t, c_t, δ_t) ($t = 1, \dots, c$) with $a_t \in \{0, 1\}$ such that for every $K \geq 0$, with ring length $L = K + 2$ and*

$$S_L = \{(a_t K + c_t)B + \delta_t : 1 \leq t \leq c\},$$

and for all $j \geq 0$, $0 \leq T < 2^L$, $0 \leq i < L$ and $q < b$:

$$((j2^L + T)L + i)B + \rho_q \in \mathcal{P}_{S_L} \iff \text{Run}_F(L, j + 1, T, i)_q = 1.$$

Thus the one-parameter family (S_L) , all of whose members have at most c moves and $\max S_L = O(L)$, runs F on the ring of length L from the initial configurations $X = 1, 2, 3, \dots$ in turn, each for 2^L steps. This theorem is formally verified (*SubGame.Univ.universal*).

Proof. We build a circuit U_F (Table 2; the complete definition is `Univ.inp` in `UnivCore.lean`, mirrored in `subgame/univ1.py`) whose delays are 0, 1, $L - 1$, L and $L + 1$, i.e. of the form $aK + c$. Write $\tau = tL + i$ and $t = j2^L + T$.

Framing. As in Lemma 9, $\text{mk}(\tau) = [i = 0]$; reading mk with delay $L - 1$ gives $\text{lc}(\tau) = [i = L - 1]$.

Timer. The bit-serial counter of Section 3 with register length L : its bit in block τ is $\text{bit}_i(t + 1)$ and its carry out of the last cell of round t is $\text{low}(t, L) = [t \equiv -1 \pmod{2^L}]$.

Run start. Hence novf at cell 0 is $[T \neq 0]$ (at $\tau = 0$ it reads boot), $\text{inj} = [i = 0 \wedge T = 0]$, and the latch $\text{rs} = \text{inj} \vee (\neg \text{mk} \wedge \text{rs}[-1])$ equals $[T = 0]$: runs are the rounds $[j2^L, (j + 1)2^L)$.

Input counter. A second counter with carry-in inj (the carry is cut at mk) is incremented once per run, during its first round; its bit in block τ is $\text{bit}_i(j + 1)$.

Cells. Let $s_q(\tau)$ be the intended value $\text{Run}_F(L, j + 1, T, i)_q$. For $T \geq 1$ the blocks $\tau - L - 1, \tau - L, \tau - L + 1$ are cells $i - 1, i, i + 1$ of round $t - 1$, i.e. time $T - 1$ of the same run; at $i = 0$ (resp. $i = L - 1$) the left (right) literal is replaced by β using mk (lc). A minterm wire is the NOR of the complemented literals, hence is 1 exactly for its pattern; $\text{nF}_q = \neg f_q(\text{neighbours})$; and the last NOR levels compute $s_q = (\neg \text{rs} \wedge f_q) \vee (\text{rs} \wedge \iota_q(u_X, \text{mk}, \text{lc}))$. When $T = 0$ the rule output is ignored, so reads from the previous run (or from negative blocks) do not matter.

The intended values therefore satisfy every NOR equation (this is `Univ.norEqns`). Finally we order the wires so that inputs with delay 0 come from earlier wires, place wire number r at coarse position $3^r - 1$ (a super-increasing, hence Sidon, sequence), and apply the General Layout Lemma (transported to arbitrary finite types of wire names); ρ_q is the residue of s_q . $\square$

Corollary 15 (Turing machines, formally verified). *Let M be a deterministic Turing machine with states Q , tape alphabet Γ and an input map $\{0, 1\} \rightarrow \Gamma$, working on the tape cells $0, \dots, L - 1$ (a move off the tape leaves the head in place). There are B , residues ρ and a fixed finite list of taps such that for every $L \geq 2$ the subtraction game S_L with moves $(a_t(L - 2) + c_t)B + \delta_t$ satisfies: for every input $X = j + 1$ (bit i on cell i), every $T < 2^L$ and every cell $i < L$, the positions $((j2^L + T)L + i)B + \rho_x$ that are P -positions are exactly those whose index x is the content of cell i (symbol, and the state if the head is there) after T steps of M on input X .*

Proof. A cell of the automaton holds either a wall (the boundary) or a symbol together with, optionally, the machine state; the rule moves the head exactly as M does, and keeps it in place when it would move into a wall. One step of this automaton is one step of M (`TM.tmRule.step`, a case analysis on the head position relative to the cell). Encoding the finitely many cell contents one-hot and applying Theorem 14 gives the claim (`TM.tm.universal`). $\square$

Two parameters. For the binary setting we also need a ring for the tape that is much longer than the ring carrying the timer and the input. The same construction with a control ring of length L_2 (timer and input counter) and a tape ring of length $L_1 = mL_2$ (the cells), plus a latch marking the first L_2 cells of each tape round and a latch that holds the timer overflow until the next tape round, gives a family S_{L_1, L_2} with a bounded number of moves and

$$((j2^{L_2} + T)L_1 + i)B + \rho_q \in \mathcal{P}_{S_{L_1, L_2}} \iff \text{Run}_F(L_1, L_2, (j + 1) \bmod 2^{L_2}, T, i)_q = 1$$

for $T < 2^{L_2}$ and $i < L_1$: the input occupies the first L_2 cells, runs last 2^{L_2} tape rounds. This two-ring circuit (57–59 wires for $b = 1$, `subgame/univ2.py`) is not formalised; we checked it as described next.

Validation. Both circuits were checked by logic simulation against a direct simulation of F , for all 256 elementary rules and for random 2-bit rules with random ι and β ($L \in \{3, 4, 5\}$; $L_1 = mL_2$ with $L_2 \leq 4$, $m \leq 3$), with no mismatch. We also built the actual subtraction games and simulated them: for rule 110 on $L = 4$ cells (123 181 moves, $B = 124\,917$) all 2.4×10^7 positions of the first three runs agree with the circuit, and for three two-ring instances ($L_2 = 3$, $L_1 = 6$, about 2.6×10^5 moves) all positions of four runs agree with the cellular automaton.

Remark 16. The number of moves c is dominated by the covering moves of the General Layout Lemma (about one per zero residue, i.e. about B), so it is large; it is nevertheless a constant, independent of L and of the initial configuration. For the counter of Section 3 exact optimisation reduced the covering from hundreds of moves to 14; we have not attempted this for U_F .

9 Complexity

Theorem 17 (PSPACE-completeness with a bounded number of moves). *The problem*

SUBGAME: *given a finite set $S \subset \mathbb{N}_{>0}$ in unary and $n \in \mathbb{N}$ in binary, is n a P -position of the subtraction game S ?*

is PSPACE-complete. There is a constant c such that it remains PSPACE-complete when restricted to $|S| \leq c$; in fact there is an explicit family $(S_L)_{L \geq 3}$ with $|S_L| \leq c$ and $\max S_L = O(L)$ such that $\{(1^L, n) : n \in \mathcal{P}_{S_L}\}$ is PSPACE-complete.

Proof. Membership. Compute $w(0), w(1), \dots, w(n)$ keeping only the last $\max S$ values and a binary counter: space $O(\max S + \log n)$.

Hardness. Let A be a PSPACE-complete language and M a deterministic single-tape machine deciding it in linear space and time $2^{O(|x|)}$. Consider the padded language $A' = \{x10^r : x \in A, r \geq 0\}$, which is PSPACE-complete as well, and a deterministic machine M' that on an input $y \in \{0, 1\}^L$ first locates the last 1 and then runs M on the prefix before it, and halts in an absorbing accepting or rejecting state. It recognises the first and last cell by marked symbols and never leaves the L cells: choosing c_1 large, when the last 1 of y is at position ℓ it uses at most $c_1(\ell + 1)$ cells and at most $2^{c_1(\ell+1)}$ steps.

Let F be the usual cellular automaton simulating M' : a cell holds a tape symbol (with the marks “first” / “last”), the machine state if the head is on it, and an *accept flag*; the rule performs one step of M' , and the new flag is the OR of the three old flags and of “the head is in the accepting state”. Let $\iota(v, \text{first}, \text{last})$ be the cell holding symbol v with the corresponding marks, the head in the initial state iff *first*, and flag 0; let β be blank. (Corollary 15 is the formal version of this simulation.) Encode the states in $b = O(1)$ bits. Let (S_L) be the one-parameter family of Theorem 14 for F ; its taps, and hence c , depend only on M .

Given x , let $y = x1$, $L = c_1(|x| + 1) + |x| + 4$, $X = \sum_i y_i 2^i$ (so $1 \leq X < 2^L$), $j = X - 1$, $T = 2^L - 1$ and $n = ((j2^L + T)L)B + \rho_{\text{flag}}$. In run j the automaton starts from y padded by zeros; M' halts after at most $2^{c_1(|x|+1)}$ steps, and afterwards the flag (if set) spreads one cell per step, so it reaches cell 0 before time $2^{c_1(|x|+1)} + L \leq 2^L - 1$. By Theorem 14, $n \in \mathcal{P}_{S_L}$ iff $x \in A$. The instance (S_L, n) is computable in polynomial time: S_L consists of c numbers of size $O(L)$, and n has $O(L)$ bits. $\square$

Theorem 18 (Binary move sets). *If the move set S is given in binary (and n in binary), deciding whether n is a P-position of S is EXPTIME-complete, and it remains EXPTIME-complete when restricted to $|S| \leq c$ for a constant c . Consequently no algorithm with running time polynomial in the input length decides the outcome of subtraction games with c moves written in binary.*

Proof. Membership. The simulation above uses space $O(\max S + \log n)$ and time $O(n|S|)$, both at most exponential in the input length.

Hardness. Let M decide an EXPTIME-complete language in time $2^{p(|x|)}$ (hence space $2^{p(|x|)}$) on a one-sided tape; as above let M' run M on the prefix of its input before the last 1, and let F simulate M' with a spreading accept flag. Use the two-parameter family S_{L_1, L_2} of Section 8.2 (two parameters) with $L_2 = p(|x|) + |x| + 4$ and L_1 the least multiple of L_2 exceeding 2^{L_2} : the input $y = x10 \cdots 0 \in \{0, 1\}^{L_2}$ is written on the first L_2 of the L_1 cells, the machine never reaches the right end of the tape, and it has 2^{L_2} steps. Take $X = \text{val}(y)$, $j = X - 1$, $T = 2^{L_2} - 1$ and $n = ((j2^{L_2} + T)L_1)B + \rho_{\text{flag}}$. In binary, S_{L_1, L_2} consists of c numbers of $O(L_2)$ bits and n has $O(L_2)$ bits, so the reduction is polynomial. The last statement follows from the time hierarchy theorem ($P \subsetneq \text{EXPTIME}$). $\square$

Thus the complexity of one-heap subtraction games is governed by the size of the largest move, not by the number of moves: they are easy for every fixed game, PSPACE-complete for a bounded number of moves written in unary, and EXPTIME-complete for a bounded number of moves written in binary.

Corollary 19 (Periods are hard to compute). *Given S in unary and p in binary, deciding whether the least outcome period of S exceeds p is PSPACE-hard.*

Proof. Here a single run suffices. Let F simulate a linear-space machine M deciding a PSPACE-complete language, with both halting states absorbing, and encode its states so that the all-zero code (blank, no head, flag 0) is quiescent. Take the circuit of Theorem 14 without the timer and the input counter, and instead OR each state bit q with $\text{OR}_{i \in I_q} \text{boot}[-i]$, where I_q is the set of cells whose initial state (on input x) has bit q equal to 1; since all reads in round 0 return 0, this loads the initial configuration, at the cost of $O(|x|)$ extra moves. After M halts the configuration is frozen, so every wire is periodic with period L blocks. Add an L -bit serial counter as in Section 3 whose carry-in is AND-ed with the accept flag of the current cell. If M rejects, the counter never moves and the eventual period divides BL ; if M accepts, the flag reaches every cell, the counter runs and, as in Theorem 10, the least period is a multiple of $BL2^L$. Take $p = BL$. $\square$

10 Discussion and open problems

A subtraction game is a shift register with a single, fixed nonlinear feedback function, and one might expect such a rigid object to be computationally weak. The results above show the opposite: with a bounded number of moves it can count to $2^{\Omega(\max S)}$, and it can simulate any space-bounded computation. Periods are long because the game can compute.

1. **How few moves?** (Open Problem 3) Our construction needs 26 moves; Flammenkamp’s data suggest that five moves may already give exponential periods. What is the least n for which periods of n -move games are not polynomially bounded? It lies between 3 and 26. Our search went $1061 \rightarrow 169 \rightarrow 54 \rightarrow 38 \rightarrow 29 \rightarrow 26$; the constraint solver’s lower bounds for fixed wire placements are far below 26, so better circuits or placements may exist.
2. **Growth rate.** Our periods grow like $2^{\max S/260}$ (or $2^{\max S/240}$ with 28 moves), much slower than the empirical $2^{0.3 \max S}$ of Flammenkamp’s records. Can one prove $2^{c \max S}$ for a larger c ? In our approach $c = 1/B$, so the question becomes how short a block can be.
3. **Complexity threshold.** Two-move games are easy and games with c moves are PSPACE-complete. What is the least number of moves for which SUBGAME is PSPACE-hard?
4. **Preperiods.** The same technique gives exponentially long preperiods (let the counter halt at its first overflow); it would be interesting to understand the joint behaviour of period and preperiod for small move sets.

Remark 20 (Any number of moves $n \geq 26$). If δ is a residue that is not a difference of two clock or wire residues (for our design, e.g. $\delta = 74$), then from every clock or wire residue a move $\equiv \delta \pmod{260}$ lands on a zero residue. Adding moves $qB + \delta$ therefore never creates a move from a P-position to a P-position, and the P-positions are unchanged. Hence for every $n \geq 26$ there are n -move games with period $260k2^k$ and largest move $O(k)$.

References

- [1] I. Althöfer and J. Bültmann, Superlinear period lengths in some subtraction games, *Theoret. Comput. Sci.* 148 (1995) 111–119.

- [2] E. R. Berlekamp, J. H. Conway and R. K. Guy, *Winning Ways for Your Mathematical Plays*, A K Peters, 2001–2004.
- [3] A. Flammenkamp, *Lange Perioden in Subtraktions-Spielen*, Dissertation, Universität Bielefeld, 1997.
- [4] U. Larsson and I. Saha, A brief conversation about subtraction games, arXiv:2405.20054 (2024); in *Games of No Chance 6*.
- [5] U. Larsson and H. Manabe, Additive subtraction games, arXiv:2603.10414 (2026).
- [6] U. Larsson and J. Wästlund, From heaps of matches to the limits of computability, *Electron. J. Combin.* 20 (2013) P41.
- [7] K. Burke, A. Dailly and N. Oijid, Complexity and algorithms for Arc-Kayles and Non-Disconnecting Arc-Kayles, arXiv:2404.10390 (2024).
- [8] V. Gurvich and M. Vyalyi, On computational hardness of multidimensional subtraction games, *Algorithms* 14 (2021) 71, arXiv:2001.03962.
- [9] U. Larsson, Impartial games emulating one-dimensional cellular automata and undecidability, arXiv:1201.1039 (2012).
- [10] I. Miklós and L. Post, Superpolynomial period lengths of the winning positions in the subtraction game, *Int. J. Game Theory* (2024), arXiv:2312.02426.
- [11] R. J. Nowakowski, Unsolved problems in combinatorial games, in *Games of No Chance 5*, MSRI Publ. 70 (2019) 125–168.
- [12] M. D. Ward, A conjecture about periods in subtraction games, arXiv:1606.04029.